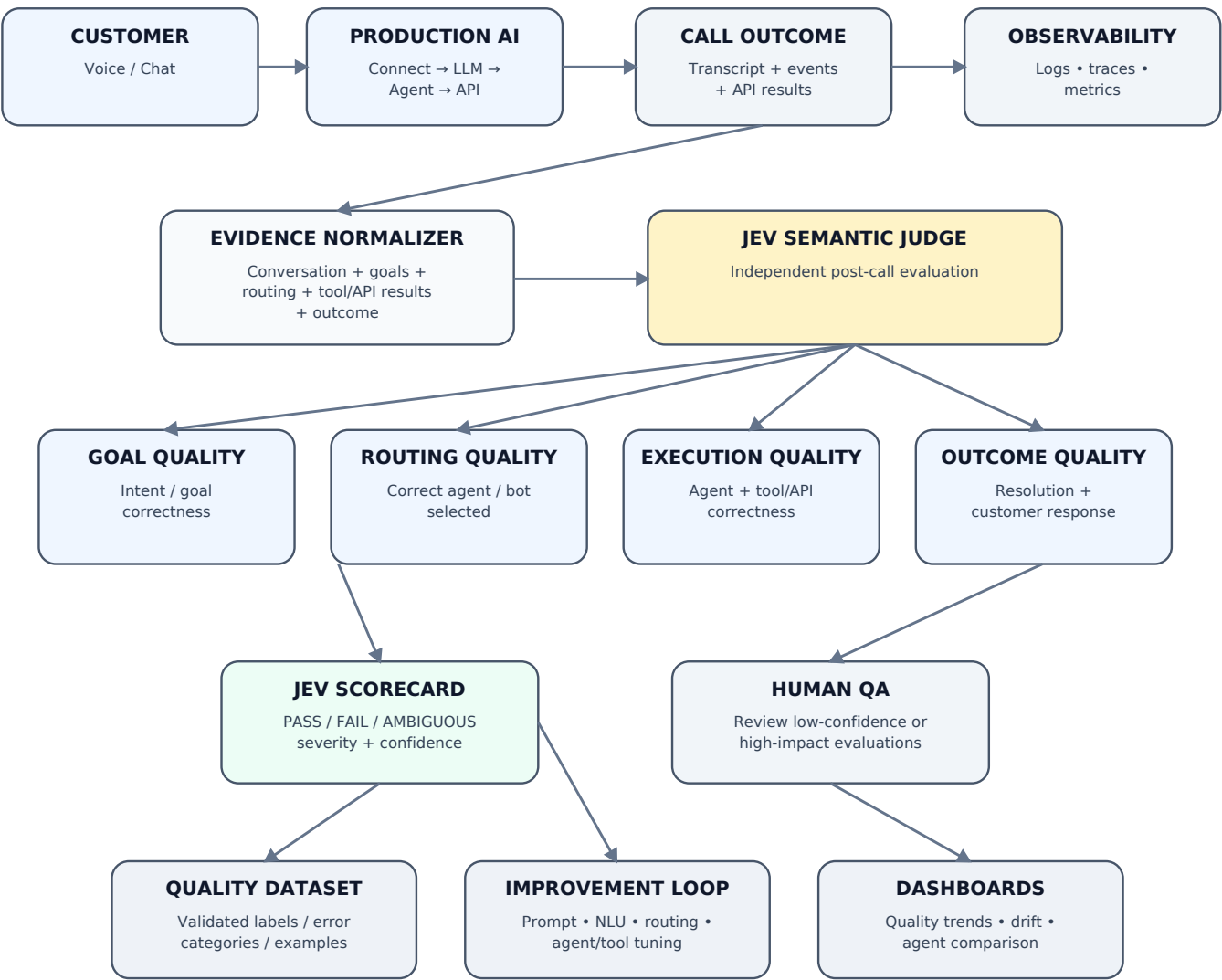


Post-Call Semantic Evaluation Framework

Using System One / JEV as a semantic judge on top of traditional observability

Presentation thesis: Traditional observability tells us what the system did. JEV evaluates whether the AI understood the customer, selected the right path, completed the goal, and communicated the outcome correctly.



Master framework: JEV is downstream of production, so the customer path has no JEV latency.

1. What JEV Evaluates

The evaluator should not simply ask “Was the response good?” It should evaluate the customer journey against structured evidence and explicit quality dimensions.

Dimension	Question JEV answers	Example failure
Goal accuracy	Did the system understand what the customer wanted?	Customer wanted a claim + rental; system recognized only rental.
Routing accuracy	Was the right specialist selected?	Roadside need routed to Service.
Goal completion	Was the requested outcome actually achieved?	Agent says claim filed; API shows creation failed.
Agent behavior	Did the agent follow the expected workflow?	Agent skipped a required verification step.
Tool/API correctness	Does the agent statement match system-of-record evidence?	Agent says reservation confirmed; API says pending.
Response accuracy	Did we tell the customer only what is supported?	Agent invents a claim status not present in data.
Customer effort	Did the customer repeat information unnecessarily?	Policy number requested twice despite being captured.
Resolution	Was the customer's overall objective resolved?	Claim solved but rental goal was dropped.
Policy adherence	Did the interaction follow required business constraints?	Action performed without required authorization.

2. Evidence Package Sent to JEV

The strongest design is to give JEV structured evidence rather than only a raw transcript. This reduces ambiguity and makes evaluations auditable.

```
{ "conversation": { "transcript": "...", "channel": "VOICE" }, "customer_goals": [ "FILE_CLAIM", "RENTAL_VEHICLE" ], "system_decisions": { "route": "RENTAL_AGENT" }, "tool_events": [ { "tool": "CreateClaim", "status": "SUCCESS" }, { "tool": "ReserveRental", "status": "PENDING" } ], "final_response": "Your claim is filed and your rental is reserved." }
```

Important: JEV evaluates the evidence; the system of record remains authoritative for transactional facts.

3. Real Example — Accident + Claim + Rental

Customer: “I was in an accident. My car isn't drivable. I need to file a claim and I need a rental car.”

Step	Production event	JEV post-call evaluation
1	Conversation model identifies RENTAL only	❑ Goal miss: FILE_CLAIM also present
2	Customer transferred to Rental Agent	❑ Routing incomplete because primary claim need was not addressed
3	Rental eligibility checked	✓ Rental workflow itself may be correct
4	Rental API succeeds	✓ Tool result matches agent outcome
5	Call ends	❑ Overall resolution = PARTIAL

JEV output

```
{ "overall": "FAIL", "severity": "HIGH", "goal_accuracy": 0.42, "routing_accuracy": 0.55, "goal_completion": "PARTIAL", "missed_goals": ["FILE_CLAIM"], "root_cause": "Multi-goal extraction/routing failure", "recommended_action": "Add labeled example to multi-goal routing dataset" }
```

This is where the framework creates value beyond operational monitoring: the call technically completed, but the customer objective was not fully resolved.

4. Example — Hallucinated Success

Customer: “Please file a claim for my accident.”

Agent response: “Your claim has been successfully filed. Your claim number is CLM12345.”

System evidence: CreateClaim API returned FAILED due to a downstream timeout.

Traditional observability

You see: Lambda invoked → API timeout → retry → call completed. Operationally visible, but semantic customer impact may not be obvious.

JEV evaluation

```
{ "agent_correct": false, "response_correct": false, "evidence_conflict": true, "severity": "CRITICAL", "customer_misinformed": true,
  "recommended_action": "Fix agent response guardrail and add failure-path test" }
```

Key presentation point: JEV detects that the system's final statement contradicts the system of record.

5. Example — Unnecessary Customer Effort

Conversation: Customer provides policy number in the first 30 seconds. Later, the Claims Agent asks for it again.

Traditional observability can show both events and their timestamps. JEV can connect them semantically and identify the repeated request as unnecessary customer effort.

Possible JEV result

```
{ "customer_effort": "POOR", "repetition_detected": true, "repeated_information": "policy_number", "avoidable_turns": 1, "root_cause": "Context not propagated to Claims Agent" }
```

This creates an actionable engineering signal: fix context propagation, rather than simply reporting a longer call duration.

6. Example — Successful Call, Wrong Outcome

Customer: “I need a rental because my vehicle is not drivable after an accident.”

The Rental Agent successfully creates a reservation. The API returns HTTP 200. The call is marked successful.

But JEV identifies that the customer also needed the accident reported and the claim was never created.

Technical outcome: SUCCESS Business outcome: PARTIAL Customer goal completion: 1 of 2 JEV: - Rental execution = PASS - Overall goal completion = FAIL - Missed goal = FILE CLAIM - Root cause = multi-goal orchestration

This distinction is critical: technical success $\neq$ customer success.

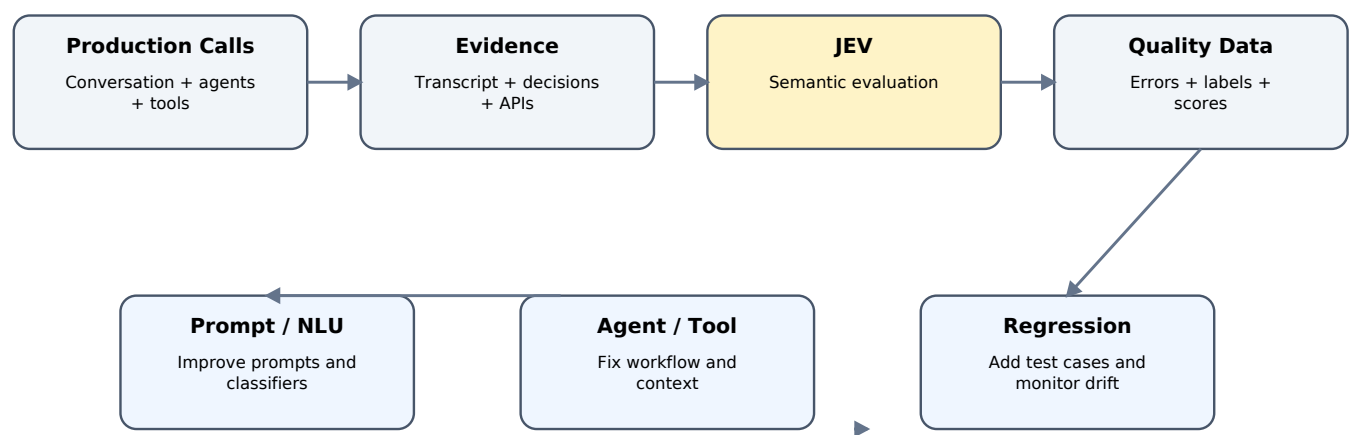
7. JEV vs Traditional Observability vs Human QA

Capability	Traditional observability	Human QA	JEV semantic evaluation
Latency monitoring	✓ Excellent	Limited	Not primary purpose
API/tool failures	✓ Excellent	Sometimes	✓ Contextual interpretation
Intent/goal correctness	Limited	✓	✓ Scalable
Routing correctness	Limited	✓	✓ Scalable
Conversation-level reasoning	Limited	✓	✓
Customer effort	Limited	✓	✓
Goal completion	Limited	✓	✓
Scale	Very high	Low/medium	High
Cost per call	Low	High	Model-dependent
Ground truth	System telemetry	Human judgment	Needs calibration
Best role	Operational truth	Gold-standard QA	Semantic quality layer

Positioning: Do not replace observability or human QA. Use JEV between telemetry and improvement as a scalable semantic evaluator, with humans validating uncertain/high-impact cases.

8. Closed-Loop AI Improvement Framework

The real enterprise value appears when JEV findings become structured improvement data.



The output should become a continuously growing evaluation and regression dataset.
Recommended feedback categories: missed goal, wrong route, unnecessary transfer, hallucinated outcome, missing context, policy deviation, unnecessary customer effort, tool/API mismatch, incomplete resolution.

9. Confidence and Human Validation

JEV should not automatically become the ground truth. Treat its evaluation as a candidate label until calibrated against human QA

JEV | review | review | | | ▼▼▼ HIGH MEDIUM LOW confidence confidence confidence | | | ▼▼▼ Auto-use Sample Human QA carefully
review review | | | ▼ Gold Dataset | ▼ JEV calibration

Track JEV-vs-human agreement, false positives, false negatives, and disagreement by intent/agent/language/channel.

10. Enterprise Metrics Dashboard

A useful dashboard should connect operational metrics to semantic customer outcomes.

Metric	Traditional telemetry	JEV semantic metric
Call completion	✓	—
Average handle time	✓	—
API success rate	✓	—
Goal accuracy	—	✓
Routing accuracy	—	✓
Goal completion	—	✓
Customer effort	—	✓
Response correctness	—	✓
Unnecessary transfer	—	✓
Agent quality	Partial	✓
Overall customer resolution	Partial	✓

11. Recommended POC

Scope: start with one journey: Auto Accident → Claim + Rental. Evaluate a historical batch first, then run JEV continuously on sampled production calls.

- Phase 1 — Baseline: collect transcript, routing, agent, API and outcome data.
- Phase 2 — JEV evaluation: score goal accuracy, routing, execution, response and resolution.
- Phase 3 — Human calibration: compare JEV with existing QA on a representative sample.
- Phase 4 — Improvement: convert high-confidence failures into labeled examples and regression tests.
- Phase 5 — Scale: extend to Service, Roadside, Agency Concierge and other journeys.

Presentation takeaway

“We don't put another model in the customer's critical path. We use JEV after the interaction to understand whether our AI actually solved the customer's problem—and turn those findings into a continuous improvement loop.”